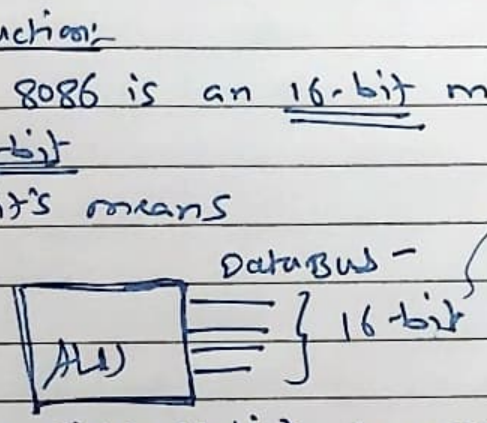
8086 microprocessor:-

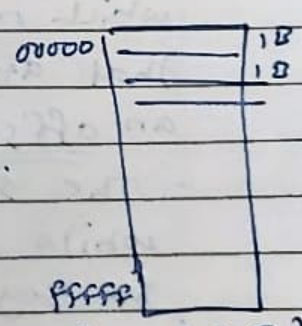
- 8086 is the successor of 8085.

- Introduction:-
- Intel 8086 is an 16-bit microprocess. whereas 8085 is 8-bit
- 16 bit's means



- At a time 16-bit is processed
- No. of operation we can performed $= 2^{16} = 65536$
- It is 40 pin integrated circuit
- Its operating freq^n are 5, 8, and 10 MHz
- It has 20-bit Address Bus
- It support pipe-line
- It has almost 29000 transistors.

8085 Operating freq^n is 3 MHz



$(2^{20} \times 1B = 1MB)$

time Unit.

	1	2	3	4	5	6
I_1	IF_1	ID_1	EX_1			
I_2		IF_2	ID_2	IF_2	ID_2	EX_2
				EX_2		

concept of pipelinging

- 8086 microprocessor works parallelly so it is divided in to two parts BIU & EU.

8086 - Architecture

- 8086 - Architecture is divided into two parts.
- i) BIU - (Bus interface Unit)
- ii) EU - (Execution Unit)

i) BIU :-

- Register :-
- 16-bit registers

- The 8086 microprocessor is an 16-bit microprocessor designed by Intel in 1970s.
- The 8086 microprocessor has a 20-bit address bus which can address upto 1MB of memory & a 16-bit data bus, which can transfer data betn microprocessor & memory or I/O device.
- The 8086 microprocessor has a segmented memory architecture which means that memory is divided into segment that are addressed using both a segment register & an offset
- The segment register points to the starting address while offset specifies the specific location within the segment.

① Register : (segment Register)

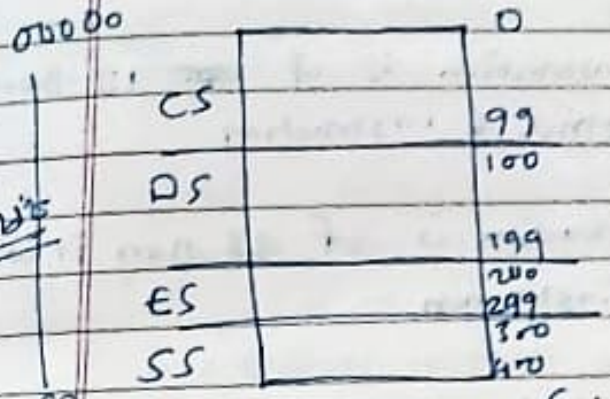
ES - Extra segment
CS - code segment
SS - stack segment
DS - Data segment

} ⇒ 16-bit each

IP - Instruction pointer Register → offset

* IP offset is used for code segment

SP is used for get the offset of stack seg
BP —"— get the offset of stack seg for finding random location
SI —"— get —"— for data seg.
DI —"— get —"— for extra seg.



starting address + offset

memory → ($1 \text{ gm } B = 2^{20}$)

- virtually divided
- All the register are having starting address
- starting address will be given by Register (ie CS, DS, ES, SS) & offset will be given by IP
- Register are of 16-bit & memory is of 20-bit

So do

case

starting offset

CS = 1A1A + 1B1B — 16 bit

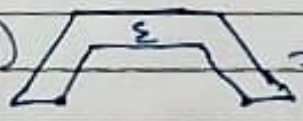
case

= 1A1A × 10H + 1B1B — 20 bit

= 1A1A0 + 1B1B

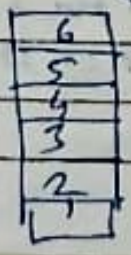
1A1A0
1B1B
1BCBB → Physical address

Physical Address

(i)  ⇒ Calculation part to calculate physical address

Physical address ⇒ form which we can fetch the instruction

(ii) Instruction stream Byte queue:



← It will pick 6-byte

I_1 - EX

I_2, I_3, I_4, I_5, I_6, I_7 — 6B

if one instruction is of ~~one~~ 1B then it will retrive 6 instruction

&

if one instruction is of 2B then it will retrive 3 instruction

ie

I_2, I_3, I_4 — 2B each

- If one instruction is of 6B then it will retrive 1 instruction
- According to the size ~~&~~ ~~bit~~ in bites no. of instruction will come into the queue.
- FIFO in queue.

iv) Instruction will be ~~fetched~~ decode through the control unit & feten into the instruction queue.

(ii) Execution Unit:

i) General purpose register: (16 bit register)

8-bit	AH	AL	AX - 16-bit
8-bit	BH	BL	BX
8-bit	CH	CL	CX
8	DH	DL	DX
←	SP		- offset Register → Used for stack segment top of stack
←	BP		offset Register
←	SI		
←	DI		

SP - offset register gives the offset for stack segment for finding top of stack.

SI & BP : offset register gives the offset for stack segment for finding random location

- SI - Offset register gives the offset for Data segment for finding particular location

- DI - Offset register for Extra segments for finding particular location

ALU: (To perform the operation)

To add two no's

10H
20H

i) mov AL, 10H
mov BL, 20H
ADD AL, BL

if no is of 8-bit use AL or AH (lower, higher)
if no is of 16-bit use AX

⇒ AL = AL + BL
AL = 10 + 20
AL = 30

ii) mov AL, 10H
ADD AL, 20H — 20H value

operands:
- It's a temporary register used for exchange instruction etc - Not used for programmer

Flag: (status)
- gives the status of your result, i.e result is +ve, -ve, parity is their or not

- Segmentation: Is the process in which main memory of the computer is divided into different segments & each segment has its own base address.
- BIU interface contains four 16-bit special purpose registers.

(i) code segment register: (16-bit reg): CS holds the base add of for the code segment. All program are stored in the code segment and accessed via the IP.

(ii) Data segment register (16 bit reg): - DS holds the base add. for the Data segment.

(iii) Stack segment: (16-bit): SS holds the base address for the stack segment

(iv) Extra segment (16 bit): ES holds the base address for the Extra segment.

Note:
- The segments are present in memory & segment registers are present in microprocessor
- segment registers store starting address of each segment in memory.

* 16-bit General purpose Registers: - Are used to store temp data within the microprocessor.

(i) AX: This is accumulator. It is 16-bit. divided into two 8-bit reg. AH & AL also perform 8-bit instructions. It is generally used for arithmetic & logical instruction
ie ADD AX, AX (AX=AX+AX)

(ii) BX: This is Base Register. It is used to store value of offset eg.) MOV BL, [500] (BL=500H)

(iii) CX: This is counter reg. It is used in looping & rotation
eg.) MOV CX, 0005
LOOP.

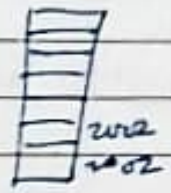
(iv) DX: This is data register. It is used in the multiplication & i/p/output port addressing
eg.) MUL BX

Physical address of 8086:

memory segmentation:

- To tal memory is divided into segments of various size.
  ↓
  an area in memory

- so in memory data is stored as bytes (8-bits) & each byte has some specific address



- 8086 has 20 lines address bus
- 8086 has 16-lines data bus

so data bus used to convert into 20-line address bus

i.e $\boxed{2^{20} \text{ bytes} = 1 \text{ MB}}$

- so address range is 00000 to FFFFF Hexadecimal value

⇒ 8086 has four different types of segment.

i) code segment ⇒ 16-bit
ii) data segment ⇒ 16-bit
iii) stack segment ⇒ 16-bit
iv) Extra segment ⇒ 16-bit

do each register the

opcode	operand

Addressing modes in 8086 microprocessor:

Defn:- ~~This specifies that the~~
- The way of specifying data to be operated by an instruction is known as addressing mode
- This specifies that the given data is an immediate data or an address. It also ~~provides~~ specifies whether the given operand is register or register pair

or:
The various formats of specifying the operands are called addressing mode

(i) Immediate addressing mode:
- In this addressing mode, data (1 byte/2 byte) specified in instruction is directly transferred into register.

ex)
mov CL, 05H, 05H will gets transferred to CL
mov BX, 1000H, 1000H will gets transferred to BX

(ii) Register Address mode:-
- In this addressing mode, operands are specified in registers only.
Here, we are not having data we are only having registers only.

ex)
mov CL, DL; content of DL will get copied into CL
mov AX, BX; BX will get copied into AX

(iii) Direct addressing mode:-

- In this addressing mode, address of operand is specified in instruction

Ex.)

mov CL, [1000H] : CL will get data from [1000H] addr

- content from 1000H will get copied into CL

4) mov AX, [1594H]
mov AL, [0300H]

mov CX, [1000H] : CL & CH will get data from 1000H & 1001H respectively.

(iv) Implied | Implicit addressing mode:-

- In this addressing mode, the operand is implied in the instruction

Ex.) STC : set the carry flag
CLD : clear Directional flag
CMA : ~~complement~~ compliment Accumulator

operand is implied implicitly inside instruction

(v) Indirect addressing mode:-

- In this, addressing mode the address of operand is store in registers.
- ~~Its~~ Address is not directly given to us. rather it is stored inside the register
- Having four different categories.
  a) Register Indirect Addressing mode
  b) Register Relative 〃 〃 〃
  c) Base index 〃 〃

d) Base Relative Plus index

a) Register indirect Addressing mode:-
- In this addressing mode, operand address will be given by memory pointer (i.e pointer may BX, SI OR DI).

Ex.)
i) mov CL, [BX], CL will take data pointed by BX address

ii) mov [BP], CL ; CL will be copied at address pointed by BP [on stack].
↳ BP always points to stack

b) Register Relative addressing mode:
- In this addressing mode, operand address will be given by memory pointer + 8-bit or 16 bit displacement.

Ex.i)
mov CL, [BX+4] ; CL will take data pointed by [BX+4] Address

c) Base indexed addressing mode:
- In this addressing mode, operand address will be given by Base Register + Index register
↳ (SI, DI, BP, SP)

Ex.i) mov CL, [BX+SI] ; CL will take data pointed by [BX+SI] address.

d) Base Relative plus indexed addressing mode:-
- In this, operand address will be given by Base Register + index register + 8-bit or 16-bit displacement

Ex.) mov CL, [BX+SI+4] ; CL will take data pointed by [BX+SI+4]

(vi) Relative addressing mode:-

- In this addressing mode, the operand address is calculated using one of the base ~~and~~ registers & an 8-bit or a 16-bit displacement.

ex:)

mov CL, [BX + 04H];

This instruction moves a byte from the address pointed by [BX+04H] in data segment ~~to~~ to CL

i.e CL ← DS: [BX+04H]

So physical address can be calculated as:

DS*10H + BX+4H

(vii) Indexed addressing mode:- (Base indexed addⁿ mode)

a)

- In this, operand address is calculated as a Base register plus an index register

i.e

Base Register: BX, BP

index Register: SI, DI

ex:)

mov CL, [BX + SI]

This instruction moves a byte from the address pointed by [BX+SI] in data segement to CL

CL ← DS: [BX+SI]

physical address calculated as [DS*10H + BX+SI]

b) Relative Base Indexed addressing mode:

- In this mode, the address of operand is calculated as the sum of Base register, index register & 8-bit or 16-bit displacement.

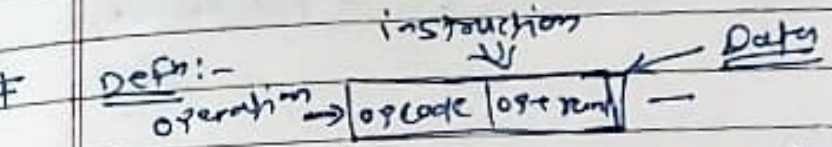
eg.) mov CL, [BX+SI+20H]

This instruction moves a byte from the address pointed by [BX+SI+20H] in data segment to CL.

CL ← DS: [BX+SI+20H]

physical address calculation ⇒ [DS*10H+BX+SI]

Defn:-



- operand:- Data: i.e on which you want to perform operation.

opcode - operation (i.e +, -, multiplication)

- In an instruction, we can't have data, instead of data it will provide address or data or register, which will be identified by addressing mode
- Data is either stored in memory or register

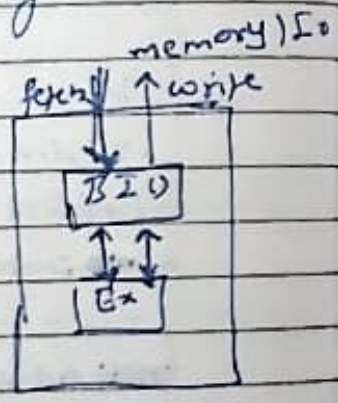
Addressing mode can reduce in instruction size by using register i.e store the data in register instead of storing it in memory

sending/receiving/storing/everything in the form of register

Divided into two independent functional unit
i) Bus Interface unit -
ii) Execution unit

- Execution unit: It is executing something.
- BIU :- Interface means acting as a interface/bridge ~~after~~ betn outer device (I/O) & processor

BIU: -
Functions:
i) Fetch the instruction or data from memory
2) write the data to memory.
or 3) write the data to ports (I/O ports)
4) Read data from ports.



so, BIU is divided into three Parts:

a) Instruction pointer (IP)
b) segment Register
c) Instruction queue

a) It is a 16-bit register. that keeps the address of memory location of comming instruction to be executed i.e Address of ~~memory location~~ next instruction to be executed.

b) segment Register:
The memory space of 1MB of 8086 is segment into 4 blocks, each block specified by register with the maximum size 64KB

1MB	CS	64KB => code segment
	DS	64KB => Data -||-
	SS	64KB => Stack -||-
	ES	64KB => Extra -||-

} 1MB

c) Instruction queue-

- BIU performs its operation in parallel with execution unit

=> BIU fetches instruction byte while execution unit is executing operation

=> The prefetched instruction is saved in group of high speed register & is known as instruction queue.